

L'utilisation de Redux

Table des matières

I. Contexte	3
II. Introduction à Redux	3
A. Présentation de Redux	3
B. Installation de Redux	4
C. Les principes de base de Redux.....	5
D. Exercice : Quiz.....	8
III. Utilisation avancée de Redux	9
A. Reducers combinés	9
B. Présentation et installation de Redux DevTools	12
C. Introduction au middleware.....	13
D. Exercice : Quiz.....	13
IV. Essentiel	14
V. Auto-évaluation	14
A. Exercice	14
B. Test.....	14
Solutions des exercices	15

I. Contexte

Durée : 1 h

Pré-requis : React , NPM , JavaScript

Environnement de travail : Navigateur Web, Visual Studio Code

Contexte

Les applications web modernes sont de plus en plus complexes, avec des états qui évoluent constamment en réponse aux actions des utilisateurs. La gestion efficace de l'état est donc devenue une préoccupation majeure pour les développeurs. C'est là qu'intervient Redux.

Redux est une bibliothèque JavaScript qui offre une solution robuste et prévisible pour la gestion de l'état dans les applications.

Dans ce cours, nous allons plonger dans le monde de Redux et découvrir comment l'installer, le configurer et l'utiliser de manière efficace. Nous allons explorer les principes de base de Redux et comprendre comment ils interagissent pour fournir une architecture de gestion de l'état cohérente et maintenable.

De plus, nous allons découvrir Redux DevTools, un outil puissant qui permet de déboguer et d'inspecter le store Redux, facilitant ainsi le processus de développement et de maintenance de nos applications.

II. Introduction à Redux

A. Présentation de Redux

Introduction aux concepts de gestion de l'état dans les applications

Lorsque nous développons des applications, qu'elles soient web ou mobiles, la gestion de l'état est l'un des aspects les plus importants à prendre en compte. L'état représente les données qui évoluent au fil du temps en réponse aux actions de l'utilisateur ou aux événements survenant dans l'application.

La gestion de l'état peut devenir complexe, en particulier dans les applications de grande envergure. Les développeurs doivent s'assurer que les différentes parties de l'application sont synchronisées et que l'état est géré de manière cohérente. Sans une gestion appropriée, l'état peut devenir difficile à maintenir, entraînant des bugs, des erreurs et une mauvaise expérience utilisateur.

C'est là qu'intervient Redux, son objectif principal est de faciliter la gestion de l'état en proposant une approche structurée et déterministe.

Qu'est-ce que Redux et pourquoi l'utiliser ?

Redux est une bibliothèque JavaScript open source, développée par Dan Abramov, qui offre une solution efficace et prévisible pour la gestion de l'état dans les applications. Bien qu'initialement conçu pour les applications React, Redux peut être utilisé avec n'importe quels framework et bibliothèque JavaScript ou dans vos applications natives.

Redux propose un modèle de gestion de l'état centralisé. L'état de l'application est stocké dans un seul endroit appelé le store, ce qui facilite la synchronisation de l'état entre les différents composants de l'application. Cela permet de maintenir un état cohérent et d'éviter les incohérences potentielles liées à la gestion de l'état local dans chaque composant.

Cette bibliothèque suit un flux de données unidirectionnel, ce qui signifie que les données circulent dans une seule direction, de manière prévisible.

En utilisant Redux, chaque modification de l'état est représentée par une action explicite. Cela permet de retracer facilement comment l'état a évolué au fil du temps et de reproduire les scénarios spécifiques en rejouant les actions. Cette traçabilité facilite le débogage et la compréhension du flux de données dans l'application.

De plus, Redux encourage le développement de code indépendant des frameworks et hautement testable. En isolant la logique de l'état dans le "store", il est plus facile de créer des tests d'intégration pour vérifier le comportement global de l'application.

Enfin, grâce à son architecture modulaire, Redux facilite le développement d'applications évolutives, permet de gérer efficacement la croissance de l'application et d'ajouter de nouvelles fonctionnalités sans introduire de complexité excessive.

Vue d'ensemble des principes fondamentaux de Redux

Pour comprendre pleinement Redux, il est essentiel de se familiariser avec ses principes fondamentaux.

Premièrement, nous avons le "store" qui est l'objet central de Redux. Il contient l'état global de l'application et offre des méthodes pour accéder à cet état, le mettre à jour et s'abonner aux changements. Le "store" est immuable, ce qui signifie que l'état ne peut être modifié que par le biais d'actions spécifiques.

Ensuite, les actions sont des objets qui décrivent une intention de modifier l'état de l'application. Elles contiennent un type qui identifie le type d'action et des données supplémentaires si nécessaire. Les actions sont généralement déclenchées par les utilisateurs ou par d'autres événements au sein de l'application.

Le dispatcher est l'un des principes fondamentaux de Redux et est responsable de la distribution des actions aux reducers. Lorsqu'une action est déclenchée, elle est envoyée au "dispatcher" qui la transmet ensuite aux reducers appropriés. Le dispatcher garantit que les actions sont traitées de manière séquentielle et ordonnée.

Par ailleurs, les reducers sont des fonctions pures qui spécifient comment l'état de l'application est modifié en réponse à une action donnée. Chaque reducer traite une action spécifique en fonction de son type et retourne une nouvelle version de l'état. Les reducers ne doivent pas modifier l'état existant, mais plutôt créer une copie modifiée de celui-ci.

Enfin, les vues sont des composants qui extraient les données nécessaires du store et les utilisent pour afficher l'interface utilisateur. Lorsque l'état du store est mis à jour, les vues sont automatiquement mises à jour pour refléter ces changements.

B. Installation de Redux

Remarque Application native

Dans ce cours, nous allons utiliser Redux dans une application native, sans utiliser de framework ou de bibliothèque tierces.

En effet, il est désormais suggéré d'utiliser Redux sous sa forme moderne, qui comprend l'utilisation d'extensions supplémentaires telles que "Redux Toolkit" pour simplifier son implémentation et son utilisation, ou encore "React-redux" dans le cas où l'on utilise Redux dans une application React.

Néanmoins, pour une compréhension complète du fonctionnement de Redux, il est préférable d'aborder son apprentissage en commençant par les fondamentaux, comme il est conseillé dans la documentation officielle.

Méthode Configuration de l'environnement de développement

Avant de pouvoir commencer à utiliser Redux, il est important de configurer correctement votre environnement de développement. Assurez-vous préalablement d'avoir Node.js installé sur votre machine. Node.js est nécessaire pour exécuter les commandes npm ou yarn et installer les dépendances nécessaires.

Commencez par créer un nouveau répertoire pour votre projet. Ensuite, ouvrez le terminal de votre éditeur de code et placez-vous dans ce répertoire.

Nous allons utiliser le bundler "Vite" pour initialiser un projet dans ce cours, mais vous pouvez choisir celui qui vous convient le mieux, comme Webpack, ou encore initialiser votre projet "from scratch".

Dans le terminal, écrivez la commande suivante pour initialiser un nouveau projet "Vite" avec npm :

```
1 npm init vite
```

Ou avec yarn :

```
1 yarn create vite
```

Nous spécifions le nom de notre projet, ensuite nous précisons que nous allons utiliser React et JavaScript.

Après avoir indiqué les différents paramètres de notre projet, celui-ci sera initialisé et contiendra l'architecture de base de Vite dans un environnement de développement natif.

Méthode Installation des dépendances Redux via npm ou yarn

Maintenant que votre projet est initié, vous pouvez installer Redux.

Nous allons accéder au dossier de notre application en écrivant la commande suivante dans le terminal de notre éditeur :

```
1 cd mon-projet
```

Ensuite, utilisez l'une des commandes suivantes dans votre terminal, en fonction de l'outil de gestion de paquets que vous utilisez :

Avec npm :

```
1 npm install redux
```

Avec yarn :

```
1 yarn add redux
```

Méthode Mise en place de la structure de base

Maintenant que vous avez installé la dépendance de Redux, vous pouvez commencer à mettre en place une structure de base pour accueillir Redux dans votre application.

Dans le répertoire "src" de votre projet, créez un dossier "redux" qui contiendra les sous-répertoires suivants : "actions", "reducers" et "store".

Le répertoire "actions" contiendra les fichiers pour vos actions spécifiques.

Dans le répertoire "reducers", nous allons créer des fichiers individuels pour chaque "reducer" que nous souhaitons mettre en place.

Enfin, dans le répertoire "store", créez un fichier "store.js" dans lequel vous allez configurer et exporter le store Redux.

C. Les principes de base de Redux

Le concept de store dans Redux

Dans Redux, le store est un objet central qui contient l'état global de l'application. Il joue un rôle essentiel dans la gestion de l'état et offre des fonctionnalités clés pour accéder à l'état, le mettre à jour et s'abonner aux changements.

Méthode Création du store

Pour créer le store dans Redux, dans le fichier "store.js", vous devez importer la fonction "createStore" fournie par Redux.

```
1 import { createStore } from 'redux'
2 const store = createStore()
```

Nous avons utilisé la fonction "createStore" pour créer le store et nous passerons notre reducer en tant que paramètre de cette fonction.

Remarque L'utilisation de createStore

En effet, ici nous utilisons createStore pour initier le store de Redux, cette méthode peut apparaître comme dépréciée, car il est recommandé d'utiliser configureStore depuis la version 4.2, il s'agit d'une nouvelle manière d'écrire du Redux, grâce à l'utilisation de Redux Toolkit. Cependant, il est essentiel de savoir mettre en place Redux avec la méthode originale avant d'apprendre à utiliser Redux Toolkit.

Utilisation des reducers

Les reducers sont des fonctions pures qui spécifient comment l'état de l'application est modifié en réponse à une action. Chaque reducer traite une action spécifique en fonction de son type et retourne une nouvelle version de l'état.

En utilisant les reducers, vous pouvez maintenir un état cohérent et prévisible dans votre application Redux. Ils permettent de séparer la logique de mise à jour de l'état de l'application du reste de la logique métier.

Méthode Création du premier reducer

Nous allons créer un fichier "counterReducer.js" dans notre dossier "reducers" et y écrire notre premier reducer :

```
1 const initialState = {
2   counter : 0,
3 }
4
5 export const counterReducer = ( state = initialState, action) => {
6   switch(action.type){
7     case 'COUNTER_INCREMENTED' :
8       return {...state, counter : state.counter + 1}
9     case 'COUNTER_DECREMENTED' :
10      return {...state, counter : state.counter - 1}
11     default : return state
12   }
13 }
```

Nous avons créé un objet "initialState" qui contient la valeur initiale de notre compteur, que nous avons passé en paramètre de notre counterReducer.

Le reducer prend en compte deux types d'actions : "COUNTER_INCREMENTED" pour incrémenter le compteur de 1 et "COUNTER_DECREMENTED" pour le décrémenter de 1. L'état est mis à jour en fonction de l'action reçue, et si l'action ne correspond à aucun des types spécifiés, l'état reste inchangé.

Notez que nous exportons notre reducer afin de pouvoir le transmettre à notre fonction "createStore" du fichier "store.js" :

```
1 import { createStore } from "redux"
2 import { counterReducer } from "../reducers/counterReducer"
3 export const store = createStore(counterReducer)
```

Méthode Accès au state

Vous pouvez maintenant accéder à l'état actuel de notre store en utilisant la méthode "getState". Cette méthode renvoie l'état actuel du store, qui peut être utilisé pour afficher les données nécessaires.

Nous allons importer notre "store" dans notre composant principal "main.js" et y accéder en utilisant cette méthode :

```
1 import { store } from '../redux/store/store'
2 const currentState = store.getState()
```

Les vues dans Redux

Dans Redux, les vues sont des composants qui extraient les données nécessaires du store et les utilisent pour afficher l'interface utilisateur.

Grâce à cette connexion, les vues peuvent accéder à l'état du store et se mettre à jour automatiquement lorsque l'état change. Cela facilite la gestion des mises à jour de l'interface utilisateur en fonction de l'état global de l'application.

Méthode Affichage du compteur dans l'application

Nous allons utiliser la valeur de l'état récupérée via la méthode "getState" pour l'afficher dans notre application. Pour cela, nous allons créer un élément "span" dans notre fichier "index.html" qui va contenir cette valeur, ainsi que deux boutons qui serviront à déclencher des événements pour modifier la valeur de notre état :

```
1 <span id="counter"></span>
2   <button id="incr">Incrémenter</button>
3   <button id="decr">Décrémenter</button>
```

Nous allons maintenant créer une fonction "render" dans notre fichier "main.js" qui va injecter la valeur de notre état dans notre span "counter".

```
1 const counterContainer = document.querySelector('#counter')
2 const render = ()=>{
3   const state = store.getState()
4   counterContainer.innerHTML = state.counter
5 }
6
7 render()
```

Nous avons désormais la valeur de notre compteur qui est affichée dynamiquement dans notre application.

Mise à jour de l'état

Pour mettre à jour le store, vous devez envoyer des actions à l'aide de la méthode "dispatch".

Une action est un objet qui décrit une intention de modifier l'état de l'application. Lorsqu'une action est dispatchée, Redux l'envoie aux reducers appropriés pour mettre à jour l'état.

Comprendre le rôle du dispatcher

Le dispatcher est un mécanisme clé dans Redux. Il reçoit les actions et les transmet aux reducers appropriés. Son rôle est de garantir que les actions sont traitées de manière séquentielle et ordonnée.

Le dispatcher est intégré au store de Redux et est utilisé automatiquement lorsque vous dispatchez une action. Vous n'avez pas besoin de le configurer manuellement.

Méthode Mettre à jour le compteur

Nous allons créer un fichier "index.js" dans notre répertoire "actions" afin de créer une action qui permettra d'incrémenter la valeur de notre compteur, ainsi qu'une action pour décrémenter cette valeur :

```
1 import { store } from "../store/store"
2 document.querySelector('#incr').addEventListener('click', ()=>{
3   store.dispatch({type : 'COUNTER_INCREMENTED'})
4 })
5
6 document.querySelector('#decr').addEventListener('click', ()=>{
7   store.dispatch({type : 'COUNTER_DECREMENTED'})
8 })
```

Tout d'abord, nous avons importé notre store, puis nous avons attribué des actions respectives à nos boutons, qui seront déclenchées au clic. Vous pouvez remarquer que nous utilisons la méthode "dispatch" de notre store pour passer l'action qui contient le type approprié pour notre reducer.

En fonction du type reçu, le switch présent dans notre counterReducer va incrémenter ou décrémenter la valeur de l'état avant de retourner sa valeur.

Méthode S'abonner au changement

Vous pouvez remarquer que nous n'avons aucun changement sur l'interface utilisateur lorsque nous incrémentons ou décrétons la valeur de notre compteur. En effet, la valeur contenue dans notre compteur n'est injectée qu'une seule fois via la fonction "render" lors de l'initialisation.

Pour remédier à cela, nous pouvons nous abonner aux changements d'état afin de mettre à jour l'interface utilisateur lorsque l'état contenu dans notre store est modifié.

Pour cela, vous pouvez écrire le code suivant dans le fichier "main.js" :

```
1 store.subscribe(render)
```

En utilisant la méthode "subscribe" de notre store, nous appelons la fonction "render" à chaque fois que l'état est mis à jour. Ainsi, nous obtenons une interface utilisateur réactive aux changements d'état.

D. Exercice : Quiz

[solution n°1 p.17]

Question 1

Qu'est-ce que le store dans Redux ?

- ☐ Un objet qui décrit une intention de modifier l'état de l'application
- ☐ Un composant qui extrait les données nécessaires du store pour afficher l'interface utilisateur
- ☐ L'objet central de Redux qui contient l'état global de l'application

Question 2

Comment peut-on modifier l'état dans Redux ?

- ☐ Directement en modifiant l'objet store
- ☐ En créant des objets actions spécifiques et en les envoyant au dispatcher
- ☐ En utilisant des fonctions pures appelées reducers

Question 3

Quelle est la responsabilité du dispatcher dans Redux ?

- ☐ Il met à jour l'interface utilisateur en fonction des changements dans le store
- ☐ Il distribue les actions aux reducers appropriés de manière séquentielle et ordonnée
- ☐ Il spécifie comment l'état de l'application est modifié en réponse à une action donnée

Question 4

Quelle est la caractéristique des reducers dans Redux ?

- ☐ Ils modifient directement l'état existant de l'application
- ☐ Ils créent une copie modifiée de l'état en réponse à une action
- ☐ Ils extraient les données nécessaires du store pour afficher l'interface utilisateur

Question 5

Qu'est-ce que les vues dans Redux ?

- ☐ Des fonctions pures qui spécifient comment l'état de l'application est modifié
- ☐ Des objets qui décrivent une intention de modifier l'état de l'application
- ☐ Des composants qui extraient les données nécessaires du store pour afficher l'interface utilisateur

III. Utilisation avancée de Redux

A. Reducers combinés

Remarque Combiner les reducers

Lorsque votre application devient plus grande et plus complexe, il peut devenir nécessaire de diviser votre état en plusieurs parties pour une gestion plus modulaire. C'est là que la combinaison de plusieurs reducers entre en jeu.

La combinaison de plusieurs reducers dans une application Redux permet de diviser la logique de gestion de l'état en plusieurs parties distinctes, appelées "sous-états" ou "sous-ensembles d'état". Chaque reducer est responsable de gérer un sous-état spécifique et expose un état partiel à l'ensemble de l'application.

Cela permet une organisation claire et une séparation des préoccupations au niveau de la gestion de l'état. Chaque reducer peut se concentrer sur une partie spécifique de l'application, ce qui facilite la maintenance et la compréhension du code.

Méthode Création d'un nouveau reducer

Nous allons créer un "reducer" qui va permettre de générer une couleur aléatoire pour la couleur de fond de notre application.

Créez un fichier "colorReducer.js" dans le dossier "reducers", ce fichier va contenir le code suivant :

```
1 const initialState = { color : 'rgb(29,29,29)' }
2 export const colorReducer = (state = initialState, action)=>{
3   switch(action.type){
4     case 'COLOR_CHANGED':
5       let randomColor = `rgb(${Math.random()* 255},${Math.random()*
6 255},${Math.random()* 255})`
7       return {...state, color : randomColor }
8     default : return state
9   }
}
```

Nous avons créé un objet "initialState" qui contient la couleur RGB initiale, que nous avons passée en paramètre de notre colorReducer.

Le reducer prend en compte un type d'action : "COLOR_CHANGED" qui génère une nouvelle couleur aléatoire et met à jour la valeur de notre état avec cette nouvelle couleur. Si l'action reçue ne correspond pas au type spécifié, l'état reste inchangé.

Méthode Création d'un nouveau bouton pour déclencher l'action

Nous allons maintenant créer un nouveau bouton dans notre fichier "index.html" afin de pouvoir déclencher par la suite un événement qui va dispatcher notre action :

```
1 <button id="randColor">Random color</button>
```

Méthode Création d'une nouvelle action

Nous pouvons désormais créer l'action qui va se déclencher lors du clic de notre nouveau bouton dans le fichier "index.js" contenu dans le dossier "actions" :

```
1 document.querySelector('#randColor').addEventListener('click', ()=>{
2   store.dispatch({type : 'COLOR_CHANGED'})
3 })
```

Lors du clic sur le bouton qui possède l'id "randColor", nous utilisons la méthode "dispatch" de notre store afin de transmettre une action contenant le type "COLOR_CHANGED".

Méthode Utilisation de "combineReducers"

Maintenant que nous avons nos deux "reducers" et leurs actions respectives, nous avons besoin d'un moyen de les transmettre à notre "store".

Actuellement, seul le reducer "counterReducer" est transmis à la méthode "createStore" de Redux.

En effet, nous devons combiner nos deux "reducers" et transmettre le résultat de cette combinaison à la méthode "createStore". Pour cela, nous pouvons créer un nouveau fichier "reducers.js" dans le dossier "reducers" qui contiendra le code suivant :

```
1 import { combineReducers } from "redux"
2 import { counterReducer } from "../counterReducer"
3 import { colorReducer } from "../colorReducer"
4
5 export const rootReducer = combineReducers({
6   counter : counterReducer,
7   color : colorReducer
8 })
```

Ici, nous avons importé la méthode "combineReducers" de Redux, ainsi que les deux reducers que nous avons créés précédemment.

Enfin, nous exportons "rootReducer" qui comprend une combinaison de nos deux reducers grâce à l'utilisation de "combineReducers".

Méthode Transmission du rootReducer à notre store

Nous pouvons maintenant importer notre "rootReducer" et le transmettre à la méthode "createStore" dans notre fichier "store.js" :

```
1 import { createStore } from "redux"
2 import { rootReducer } from "../reducers/reducers"
3 export const store = createStore(rootReducer)
```

Méthode Mise à jour de l'interface utilisateur

Nous avons désormais accès à la couleur contenue dans notre store via la méthode “getState” dans le fichier “main.js”.

Dans notre fonction “render”, nous pouvons attribuer cette couleur à la propriété CSS “backgroundColor” du body de notre page :

```
1 const counterContainer= document.querySelector('#counter')
2 const body = document.querySelector('body')
3
4 const render = ()=>{
5   const { color, counter } = store.getState()
6   counterContainer.innerHTML = counter.counter
7   body.style.backgroundColor = color.color
8 }
```

Vous pouvez noter que nous avons utilisé le destructuring afin d'extraire l'objet “color” ainsi que l'objet “counter” qui sont les sous-ensembles d'état de notre store, puis nous mettons à jour les éléments de l'interface utilisateur avec les valeurs récupérées.

Désormais, la couleur initiale de notre état est attribuée au background du body, et est modifiée à chaque clic sur le bouton.

Charge utile d'une action

Dans Redux, une action est un objet JavaScript qui décrit un événement qui s'est produit dans l'application. Le principe de payload se rapporte à une propriété spécifique de l'objet action appelée “payload”.

Le payload est généralement utilisé pour transporter des données supplémentaires avec une action. Il contient les informations pertinentes associées à l'action en cours, telles que les données à mettre à jour dans le store Redux.

Méthode Transmettre la couleur en tant que payload

Nous pouvons transmettre la couleur générée aléatoirement en tant que charge utile de notre action, pour cela nous pouvons ajouter le code suivant dans notre fichier “index.js” contenu dans le dossier “actions” :

```
1 document.querySelector('#randColor').addEventListener('click', ()=>{
2   let randomColor = `rgb(${Math.random()* 255},${Math.random()* 255},${Math.random()* 255})`
3   store.dispatch({type : 'COLOR_CHANGED',payload : randomColor})
4 })
```

Ici nous passons la variable “randomColor” qui contient notre couleur aléatoire dans la propriété “payload” de notre action.

Méthode Utilisation de la charge utile dans le reducer

Nous pouvons maintenant attribuer la charge utile de notre action dans notre colorReducer pour mettre à jour l'état :

```
1 export const randomColorReducer = (state = initialState, action)=>{
2   switch(action.type){
3     case 'COLOR_CHANGED':
4       return {...state, color : action.payload }
5     default : return state
6   }
7 }
```

B. Présentation et installation de Redux DevTools

Vue d'ensemble des fonctionnalités de Redux DevTools

Redux DevTools est un ensemble d'outils de développement qui facilite le débogage, l'inspection et la surveillance de votre application Redux. Il offre des fonctionnalités avancées pour comprendre et analyser le flux des actions, ainsi que l'état de votre store Redux.

Redux DevTools permet de remonter dans le temps et de rejouer toutes les actions qui ont modifié l'état de votre application. Cela vous permet de visualiser l'historique des actions et de revenir à des états précédents, ce qui est extrêmement utile pour le débogage et la reproduction des bugs.

Vous pouvez également inspecter chaque action dispatchée dans votre application, afficher les détails de l'action, y compris son type et sa charge utile (payload). Cela vous permet de comprendre comment les actions modifient l'état de votre application.

Visualisation de l'état : Redux DevTools offre une visualisation graphique de l'état de votre application à chaque instant. Vous pouvez voir comment l'état évolue au fil du temps et repérer facilement les changements importants.

Méthode Installation de Redux DevTools dans l'application

Pour installer Redux DevTools, vous pouvez ajouter une extension compatible à votre navigateur. Redux DevTools est disponible en tant qu'extension pour les navigateurs tels que Chrome, Firefox et autres.

Il est également possible d'installer Redux DevTools en utilisant de nombreuses alternatives proposées dans la documentation officielle de Redux DevTools :

reduxjs / redux devtools¹

Ces alternatives peuvent varier selon le navigateur que vous utilisez.

Méthode Configuration de Redux DevTools

Nous pouvons configurer Redux DevTools en passant un argument supplémentaire à notre fonction "createStore" :

```
1 import { createStore } from 'redux';
2 import rootReducer from './reducers';
3
4 const store = createStore(
5   rootReducer,
6   window.__REDUX_DEVTOOLS_EXTENSION__ && window.__REDUX_DEVTOOLS_EXTENSION__()
7 );
```

Dans cet exemple, la fonction "createStore" est appelée avec deux arguments.

Le premier argument est le rootReducer, et le deuxième argument utilise la méthode "REDUX_DEVTOOLS_EXTENSION" pour activer Redux DevTools lors de la création du store.

Il est possible de personnaliser la configuration de cet outil en fonction des besoins spécifiques de votre application. L'ensemble des techniques de configuration est disponible dans la documentation officielle de Redux DevTools.

¹ <https://github.com/reduxjs/redux-devtools/tree/main/extension#installation>

Utilisation des outils de développement pour le débogage et l'inspection du store

Une fois Redux DevTools configuré, vous pouvez utiliser les outils de développement pour déboguer et inspecter votre store Redux.

En ouvrant l'extension Redux DevTools dans votre navigateur, vous pouvez accéder à toutes les fonctionnalités mentionnées précédemment. Vous pouvez naviguer dans l'historique des actions, examiner les détails des actions individuelles, visualiser l'état à différents moments et même effectuer des modifications pour tester votre application.

Cela facilite grandement le débogage des problèmes d'état et l'inspection de l'interaction entre les actions et l'état de votre application.

En conclusion, Redux DevTools est un outil puissant pour le développement et le débogage d'applications utilisant Redux. Il offre des fonctionnalités avancées pour inspecter, analyser et déboguer votre store Redux.

C. Introduction au middleware

Qu'est qu'un middleware ?

Dans le contexte de Redux, un middleware est une fonction qui agit comme une couche intermédiaire entre les actions que vous envoyez à Redux et le moment où elles atteignent le reducer. Il intervient dans le flux de données de Redux et permet d'ajouter des fonctionnalités supplémentaires aux actions, telles que la gestion asynchrone, la journalisation ou la manipulation des actions avant qu'elles n'atteignent le reducer.

Concrètement, lorsque vous envoyez une action à Redux à l'aide de la fonction dispatch, le middleware a la possibilité de l'intercepter avant qu'elle ne soit traitée par le reducer. Il peut alors effectuer des opérations sur cette action, comme la transformation, l'annulation ou la création de nouvelles actions. Il peut également exécuter des tâches asynchrones, telles que des appels réseau, avant de relâcher l'action modifiée ou une autre action.

D. Exercice : Quiz

[solution n°2 p.18]

Question 1

Que permet de faire Redux DevTools en ce qui concerne les actions ?

- ☐ Il permet de remonter dans le temps et de rejouer les actions passées
- ☐ Il permet de créer de nouvelles actions dans l'application
- ☐ Il permet de limiter le nombre d'actions pouvant être dispatchées

Question 2

Quelles informations pouvez-vous inspecter pour chaque action dispatchée dans votre application à l'aide de Redux DevTools ?

- ☐ Uniquement le type de l'action
- ☐ Uniquement la charge utile (payload) de l'action
- ☐ Le type et la charge utile (payload) de l'action

Question 3

Quel est le rôle du payload dans Redux ?

- ☐ Transporter des données supplémentaires avec une action
- ☐ Définir les règles de validation des actions dans Redux
- ☐ Exécuter des fonctions asynchrones dans Redux

Question 4

Quelle est la fonctionnalité principale de `combineReducers` dans Redux ?

- ☐ Combiner plusieurs réducteurs en un seul
- ☐ Diviser un réducteur en plusieurs parties
- ☐ Créer un nouveau réducteur à partir de zéro

Question 5

Quelles tâches un middleware peut-il effectuer sur une action ?

- ☐ La transformation, l'annulation ou la création de nouvelles actions
- ☐ La suppression de l'action du flux de données
- ☐ L'exécution de tâches de rendu côté serveur

IV. Essentiel

Au terme de ce cours, nous avons exploré les principes fondamentaux ainsi que les concepts clés de la gestion de l'état. Nous avons compris la puissance de Redux en créant un store, un conteneur central pour l'état, et en utilisant des reducers pour définir l'évolution de l'état en réponse aux actions. De plus, nous avons examiné la manière dont les vues fonctionnent dans Redux, en affichant l'état dans notre application et en le mettant à jour de manière contrôlée à l'aide du dispatcher.

En combinant plusieurs reducers, nous avons pu organiser notre logique en modules autonomes et réutilisables. Nous avons découvert que l'utilisation de Redux DevTools peut s'avérer être précieuse pour le débogage et l'inspection du store, facilitant ainsi le développement d'applications Redux.

Ce cours offre des bases solides pour maîtriser et intégrer efficacement Redux dans nos projets JavaScript. Cependant, il est essentiel de continuer à pratiquer et à explorer les fonctionnalités avancées que Redux offre.

V. Auto-évaluation

A. Exercice

En se basant sur notre application actuelle, imaginons avoir besoin d'une nouvelle fonctionnalité qui permette d'incrémenter notre compteur d'un nombre entier aléatoire entre 0 et 10.

Question 1

[solution n°3 p.19]

Créer un nouveau bouton qui génère une nouvelle action au clic qui envoie un nombre entier aléatoire entre 0 et 10 à notre reducer, le reducer doit interpréter l'action et mettre à jour notre compteur.

Question 2

[solution n°4 p.19]

Utilisez l'outil Redux DevTools afin de s'informer sur la valeur du nombre aléatoire reçue par la charge utile de notre action et rétablir l'état de notre store à l'état précédent l'action.

B. Test

Exercice 1 : Quiz

[solution n°5 p.20]

Question 1

Qu'est-ce qu'une action dans Redux ?

- ☐ Un objet JavaScript qui décrit un événement dans l'application
- ☐ Une fonction JavaScript qui met à jour le store Redux
- ☐ Un élément HTML qui déclenche un événement dans l'application

Question 2

Il n'existe pas d'autres alternatives que l'utilisation des extensions de navigateur pour utiliser Redux DevTools.

- ☐ Vrai
- ☐ Faux

Question 3

Qu'est-ce que le payload dans une action Redux ?

- ☐ Une propriété spécifique de l'objet action
- ☐ Une fonction JavaScript utilisée pour transporter des données
- ☐ Un élément HTML associé à une action

Question 4

Il est avantageux d'utiliser combineReducers par rapport à avoir un seul réducteur géant dans une application Redux.

- ☐ Vrai
- ☐ Faux

Question 5

Qu'est-ce qu'un middleware dans le contexte de Redux ?

- ☐ Une fonction qui agit comme une couche intermédiaire entre les actions et le réducteur
- ☐ Un outil de débogage utilisé pour inspecter les actions
- ☐ Une bibliothèque tierce pour la gestion de l'état dans Redux

Solutions des exercices

Exercice p. 8 Solution n°1**Question 1**

Qu'est-ce que le store dans Redux ?

- ☐ Un objet qui décrit une intention de modifier l'état de l'application
- ☐ Un composant qui extrait les données nécessaires du store pour afficher l'interface utilisateur
- ☒ L'objet central de Redux qui contient l'état global de l'application
-  Le store est l'objet central qui contient l'état global de l'application. C'est une structure de données qui stocke l'état de votre application à un moment donné. Le store est unique et accessible depuis n'importe quelle partie de votre application.

Question 2

Comment peut-on modifier l'état dans Redux ?

- ☐ Directement en modifiant l'objet store
- ☒ En créant des objets actions spécifiques et en les envoyant au dispatcher
- ☐ En utilisant des fonctions pures appelées reducers
-  Dans Redux, l'état est modifié en créant des objets actions qui décrivent les changements à effectuer, puis en les envoyant au dispatcher. Le dispatcher est responsable de transmettre ces actions aux fonctions de réduction appropriées, appelées reducers. Les reducers sont des fonctions pures qui prennent l'état actuel et l'action comme arguments, et retournent le nouvel état modifié.

Question 3

Quelle est la responsabilité du dispatcher dans Redux ?

- ☐ Il met à jour l'interface utilisateur en fonction des changements dans le store
- ☒ Il distribue les actions aux reducers appropriés de manière séquentielle et ordonnée
- ☐ Il spécifie comment l'état de l'application est modifié en réponse à une action donnée
-  La responsabilité du dispatcher dans Redux est de distribuer les actions aux reducers appropriés de manière séquentielle et ordonnée. Le dispatcher reçoit les actions créées par l'interface utilisateur ou d'autres sources, puis les transmet aux reducers qui vont traiter ces actions et mettre à jour le store en conséquence.

Question 4

Quelle est la caractéristique des reducers dans Redux ?

- ☐ Ils modifient directement l'état existant de l'application
- ☒ Ils créent une copie modifiée de l'état en réponse à une action
- ☐ Ils extraient les données nécessaires du store pour afficher l'interface utilisateur
-  Les reducers dans Redux sont des fonctions pures qui prennent l'état actuel de l'application et une action comme paramètres, et retournent une nouvelle copie de l'état modifié en fonction de cette action. Les reducers ne modifient jamais directement l'état existant de l'application, mais plutôt créent une nouvelle copie de l'état avec les modifications nécessaires.

Question 5

Qu'est-ce que les vues dans Redux ?

- ☐ Des fonctions pures qui spécifient comment l'état de l'application est modifié
- ☐ Des objets qui décrivent une intention de modifier l'état de l'application
- ☒ Des composants qui extraient les données nécessaires du store pour afficher l'interface utilisateur
-  Dans Redux, les vues se réfèrent aux composants de l'interface utilisateur qui extraient les données nécessaires du store pour les afficher. Ces composants sont généralement responsables de la présentation des données et de l'interaction avec l'utilisateur.

Exercice p. 13 Solution n°2

Question 1

Que permet de faire Redux DevTools en ce qui concerne les actions ?

- ☒ Il permet de remonter dans le temps et de rejouer les actions passées
- ☐ Il permet de créer de nouvelles actions dans l'application
- ☐ Il permet de limiter le nombre d'actions pouvant être dispatchées
-  Redux DevTools permet de remonter dans le temps et de rejouer les actions passées. Cela signifie que vous pouvez revenir en arrière et revoir les actions précédemment dispatchées, ainsi que l'état de l'application à chaque étape.

Question 2

Quelles informations pouvez-vous inspecter pour chaque action dispatchée dans votre application à l'aide de Redux DevTools ?

- ☐ Uniquement le type de l'action
- ☐ Uniquement la charge utile (payload) de l'action
- ☒ Le type et la charge utile (payload) de l'action
-  Redux DevTools fournit un aperçu des actions dispatchées dans votre application Redux, ce qui facilite le suivi de l'état de votre application et le débogage. L'outil affiche le type de l'action ainsi que la charge utile associée à chaque action.

Question 3

Quel est le rôle du payload dans Redux ?

- ☒ Transporter des données supplémentaires avec une action
- ☐ Définir les règles de validation des actions dans Redux
- ☐ Exécuter des fonctions asynchrones dans Redux
-  Le payload est utilisé pour transporter des informations spécifiques nécessaires pour effectuer une action. Lorsqu'une action est déclenchée, elle est envoyée aux reducers de Redux, qui sont responsables de mettre à jour l'état global de l'application en fonction de l'action reçue. Les reducers peuvent accéder au payload de l'action pour obtenir les données nécessaires afin d'effectuer les modifications appropriées.

Question 4

Quelle est la fonctionnalité principale de combineReducers dans Redux ?

- ☒ Combiner plusieurs réducteurs en un seul
- ☐ Diviser un réducteur en plusieurs parties
- ☐ Créer un nouveau réducteur à partir de zéro
-  La fonctionnalité principale de `combineReducers` dans Redux est de combiner plusieurs réducteurs individuels en un seul réducteur global. Dans une application Redux, vous pouvez avoir plusieurs réducteurs qui gèrent différentes parties de l'état global. `combineReducers` permet de combiner ces réducteurs en un seul réducteur racine qui peut être utilisé par le store Redux.

Question 5

Quelles tâches un middleware peut-il effectuer sur une action ?

- ☒ La transformation, l'annulation ou la création de nouvelles actions
- ☐ La suppression de l'action du flux de données
- ☐ L'exécution de tâches de rendu côté serveur
-  Le middleware peut intercepter une action avant qu'elle ne soit traitée par le système et effectuer des opérations telles que la transformation des données de l'action, l'annulation de l'action ou la création de nouvelles actions basées sur l'action d'origine.

p. 14 Solution n°3

Nous pouvons d'abord créer le nouveau bouton dans le fichier "index.html" :

```
1 <button id="randomIncr">Random Increment</button>
```

Créer un nouvel événement qui va générer le nombre aléatoire et envoyer une nouvelle action à notre reducer :

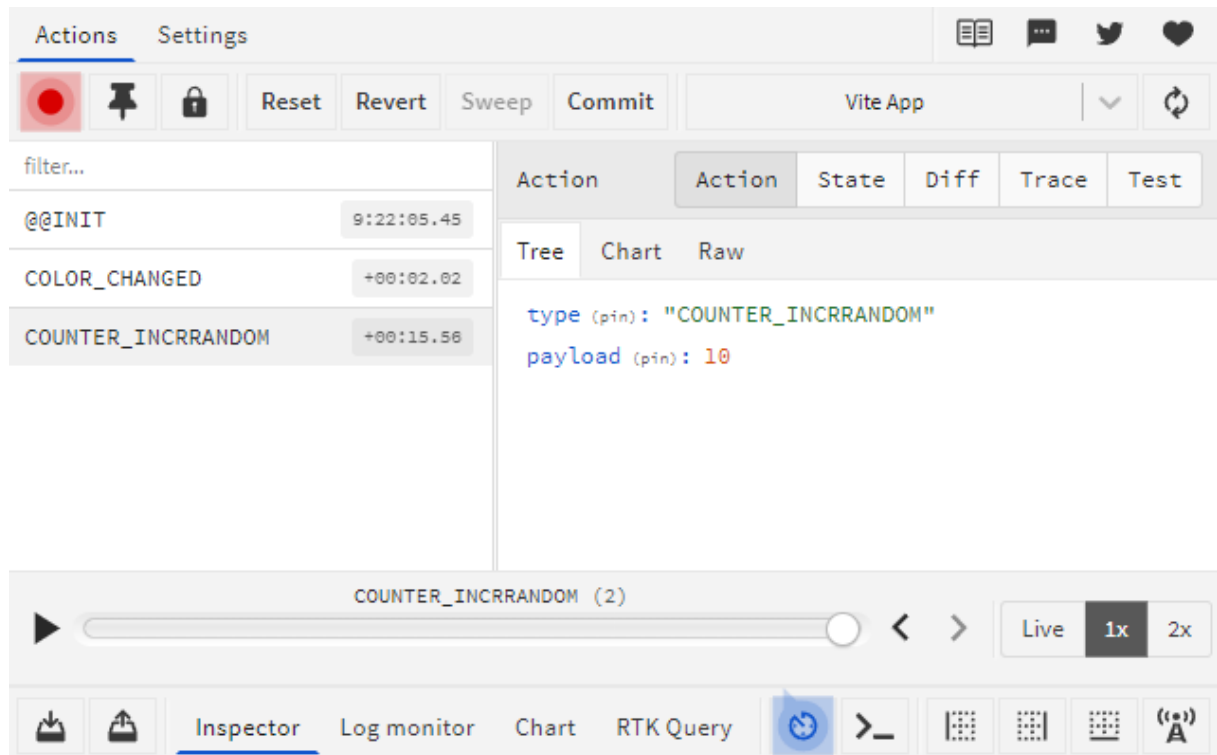
```
1 document.querySelector('#randomIncr').addEventListener('click', () => {
2   let randomNumber = Math.round(Math.random() * 10);
3   store.dispatch({ type: 'COUNTER_INCRRANDOM', payload: randomNumber });
4 });
```

Nous pouvons désormais réceptionner l'action dans le reducer, afin de la traiter et modifier la valeur de l'état :

```
1 export const counterReducer = (state = initialState, action) => {
2   switch (action.type) {
3     case 'COUNTER_INCREMENTED':
4       return { ...state, counter: state.counter + 1 };
5     case 'COUNTER_DECREMENTED':
6       return { ...state, counter: state.counter - 1 };
7     case 'COUNTER_INCRRANDOM':
8       return { ...state, counter: state.counter + action.payload };
9     default:
10      return state;
11   }
12};
```

p. 14 Solution n°4

Après avoir incrémenter le compteur avec notre nouveau bouton, nous pouvons accéder à l'outil Redux DevTools afin d'inspecter l'action qui vient d'être transmise à notre reducer :



Ici, nous pouvons observer que la charge utile de notre action a transmis un nombre qui a une valeur de 10.

Nous pouvons retourner à l'état initial de notre application en appuyant sur le bouton reset, nous pouvons également nous déplacer à un moment précis de l'application en sélectionnant une action spécifique puis sur le bouton "Jump" pour nous transporter au moment où cette action vient de s'exécuter.

Exercice p. 14 Solution n°5

Question 1

Qu'est-ce qu'une action dans Redux ?

- ☒ Un objet JavaScript qui décrit un événement dans l'application
- ☐ Une fonction JavaScript qui met à jour le store Redux
- ☐ Un élément HTML qui déclenche un événement dans l'application
- ☒ Dans Redux, une action est un objet JavaScript qui représente un événement qui s'est produit dans l'application. Il contient une propriété obligatoire appelée "type", qui indique le type d'action qui s'est produit. Les actions peuvent également contenir d'autres données pertinentes.

Question 2

Il n'existe pas d'autres alternatives que l'utilisation des extensions de navigateur pour utiliser Redux DevTools.

- ☐ Vrai
- ☒ Faux

-  Il existe plusieurs alternatives à l'utilisation des extensions de navigateur pour utiliser Redux DevTools. Les extensions de navigateur sont une option populaire et pratique, mais elles ne sont pas la seule façon d'accéder à ces outils de développement pour Redux.

Question 3

Qu'est-ce que le payload dans une action Redux ?

- ☒ Une propriété spécifique de l'objet action
- ☐ Une fonction JavaScript utilisée pour transporter des données
- ☐ Un élément HTML associé à une action

-  Le payload dans une action Redux se réfère à la propriété spécifique de l'objet action qui contient les données ou les informations pertinentes à transmettre au store Redux.

Question 4

Il est avantageux d'utiliser combineReducers par rapport à avoir un seul réducteur géant dans une application Redux.

- ☒ Vrai
- ☐ Faux

-  L'avantage d'utiliser combineReducers dans une application Redux est qu'il facilite la division des responsabilités de gestion de l'état. Redux encourage à découper la logique de l'application en plusieurs réducteurs (reducers), chaque réducteur étant responsable d'une partie spécifique de l'état global. Cela permet d'organiser le code de manière modulaire et de mieux gérer la complexité de l'application.

Question 5

Qu'est-ce qu'un middleware dans le contexte de Redux ?

- ☒ Une fonction qui agit comme une couche intermédiaire entre les actions et le réducteur
- ☐ Un outil de débogage utilisé pour inspecter les actions
- ☐ Une bibliothèque tierce pour la gestion de l'état dans Redux

-  Dans le contexte de Redux, un middleware est une fonction qui est placée entre les actions et les réducteurs. Il intercepte les actions avant qu'elles n'atteignent les réducteurs et permet d'effectuer des opérations supplémentaires, telles que la gestion des effets secondaires, la journalisation, la transformation des actions, etc.